

Formal Quantum Validation

From Qiskit programs to executable validation
and machine-checked quantum circuits

Introduction to Quantum Computing
Leonardo Chiaretti

From Research Question to Project

Quantum software introduces validation problems that are difficult to address with classical testing alone:

- measurements provide only partial information;
- measurement modifies the quantum state;
- quantum programs are intrinsically probabilistic;
- relative phase is not directly visible in computational-basis measurements; its effects emerge through interference
- compilation can substantially transform a circuit.

Research question

Can practical Qiskit circuits be connected to formal specifications and machine-checked proofs?

This project is an experimental answer to that question.

Why Testing Is Not Enough

“Program testing can be used to show the presence of bugs, but never to show their absence!”
- Dijkstra

Testing provides evidence that a property holds for selected executions: $P(x_1), P(x_2), \dots, P(x_k)$

Example of a universal correctness property: $\forall x \in D, P(x)$

Meaning: for every input x belonging to the domain D , the property P holds.

For quantum software the distinction is especially important:

- finite-shot measurements give statistical evidence;
- simulation examines concrete circuit instances;
- equal probabilities do not necessarily imply equal quantum states;

The Key Example: Relative Phase

Consider two Bell states:

$$|\Phi^+\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}$$
$$|\Phi^-\rangle = \frac{|00\rangle - |11\rangle}{\sqrt{2}}$$

In the computational basis, both give:
 $\Pr[00] = \Pr[11] = 1/2$

But they are different quantum states.

The difference is a relative phase, which affects future interference.

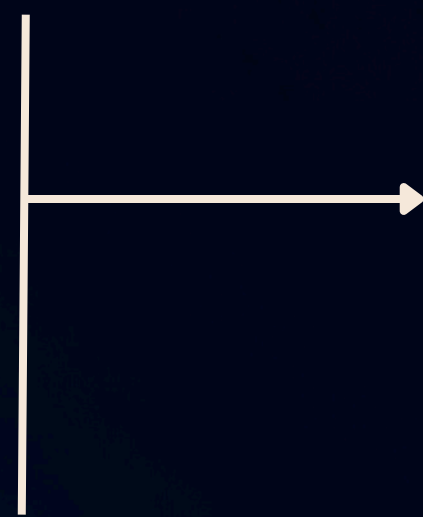
Measurement statistics in one basis are therefore insufficient to characterize semantic correctness.

Global phase is physically irrelevant; relative phase is not.

Same Statistics, Different Quantum States

State A

$$|\Phi+\rangle = (|00\rangle + |11\rangle) / \sqrt{2}$$



Computational-basis measurement

For both states:

$$\text{Pr}[00] = 1/2$$

$$\text{Pr}[11] = 1/2$$

State B

$$|\Phi-\rangle = (|00\rangle - |11\rangle) / \sqrt{2}$$

However:

The relative phase is different.

After applying $H \otimes H$:

$$|\Phi+\rangle \rightarrow (|00\rangle + |11\rangle) / \sqrt{2}$$

$$|\Phi-\rangle \rightarrow (|01\rangle + |10\rangle) / \sqrt{2}$$



The same measurement distribution in one basis does not imply the same quantum state.



Executable demonstration: fqv-demo-phase
Computational-basis probabilities: PASS
Exact quantum state: FAIL

Project Goal

The project connects a restricted quantum circuit representation to four different levels of validation:

1. structural validation
2. executable Qiskit contracts
3. transpilation validation: numerical operator equivalence + certified self-inverse cancellation
4. machine-checked proofs

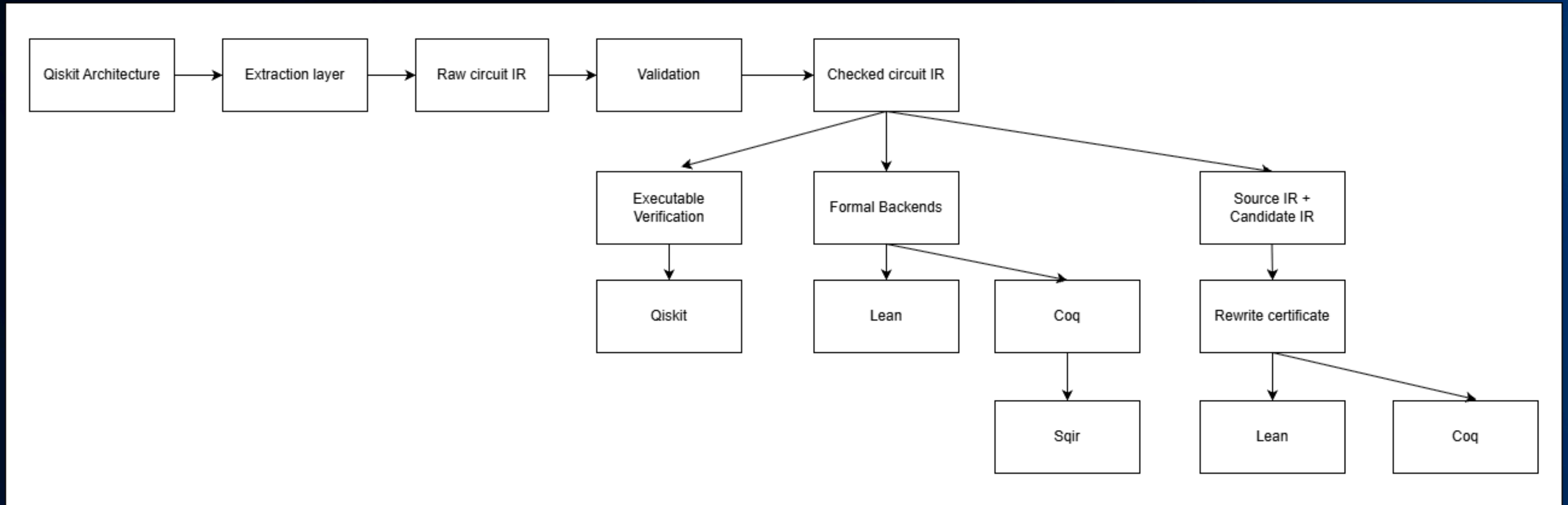
Two independent formal backends are used:

- Lean 4
- Coq + SQIR / QuantumLib

The objective is not to build a complete quantum verification framework.

The objective is to build a **small, understandable end-to-end prototype in which every boundary is explicit.**

Overall Architecture



Why Introduce an Intermediate Representation?

Directly translating arbitrary Python or Qiskit code into a theorem prover would create a very large trusted boundary.

Instead, the project defines a small Circuit IR that decouples the frontend from the formal backends

Supported gates:

- I
- X
- Z
- H
- CNOT
- SWAP

The IR explicitly fixes:

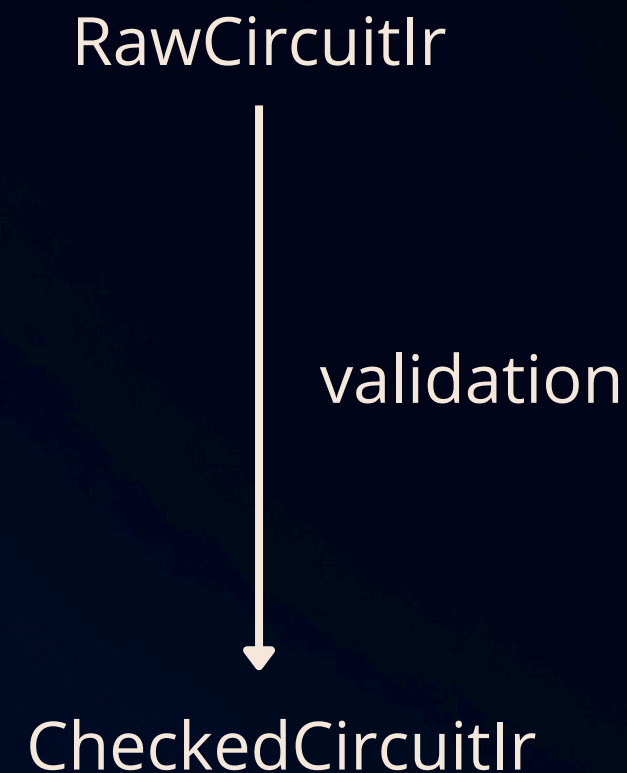
- qubit count;
- gate names;
- operands;
- execution order;
- bit-order convention.

Raw JSON is considered untrusted; formal generation begins only after validation.

IR 0.1 is intentionally restricted to sequential unitary circuits

Raw IR and Checked IR

The Python architecture distinguishes:



Validation checks:

- supported gate names;
- correct gate arity;
- qubit indices within bounds;
- distinct operands for CNOT and SWAP;
- valid circuit dimension;
- preserved operation order.

This distinction is important:

Parsing data is not the same as proving that the data satisfies the structural assumptions expected by later stages.

The checked IR is immutable and becomes the internal representation consumed by the backends.

Contracts: What Should the Circuit Do?

The circuit and its specification are deliberately separated.
A contract can describe:

Structural properties

- number of qubits;
- gate inventory;
- maximum gate count.

Input

- exact initial state.

Output

- expected final state.

Observable behaviour

- expected probabilities;
- sampled probability tolerances.

Conceptually:

Circuit IR = what is executed

Contract = what should be true

This allows the same circuit to be checked against different specifications.

Executable Validation with Qiskit

The executable backend reconstructs a Qiskit circuit from the checked IR.

It then checks several independent properties:

- circuit structure;
- state-vector comparison against the specified target;
- selected ideal probabilities;
- sampled measurement probabilities;
- resource constraints.

These checks intentionally answer different questions.

For example:

state-vector equality

can detect a relative-phase error that

computational-basis probabilities

may completely miss.

Executable validation gives strong evidence, but it remains evaluation of concrete instances.

Validating Transpilation

A second executable problem concerns compilation.

Starting from:

Source circuit



Qiskit transpiler



Transpiled circuit

We want to check:

$\mathbf{U_source} \simeq e^{i\phi} \mathbf{U_transpiled}$ using Qiskit/NumPy, up to global phase.

The project compares the complete operators, not only their action on one chosen input.

Certified PoC:

If the transformation can be explained entirely by **cancellations of adjacent equal self-inverse gates**:

$G; G \rightarrow I, \quad G \in \{X, Z, H, \text{CNOT}, \text{SWAP}\}$
generate an exact Lean/Coq proof obligation.

The Lean Formal Model

Lean provides a native finite-state semantics.

For an arbitrary number of qubits:

Basis $n := \text{Fin } n \rightarrow \text{Bool}$

State $n := \text{Basis } n \rightarrow \mathbb{C}$

A basis element assigns a Boolean value to every qubit.

A quantum state assigns a complex amplitude to every basis element:

$$|\psi\rangle = \sum_b \psi(b) |b\rangle$$

Gate operands use bounded indices.

For binary gates, Lean additionally carries a proof that the two operands are distinct (defence in depth).

Therefore many malformed circuits cannot even be represented inside the formal semantics.

Lean State Representation

GeneralCircuit.lean

```

21 namespace QuantumValidation
22 namespace General
23
24 open Complex
25
26 /-- One computational-basis assignment for an `n`-qubit register. -/
27 abbrev Basis (n : Nat) := Fin n → Bool
28
29 /-- An `n`-qubit pure state maps each basis assignment to one amplitude. -/
30 abbrev State (n : Nat) := Basis n → ℂ

```

Circuit.lean

```

16
17 /-- A two-qubit pure state in Qiskit's little-endian basis order. -/
18 abbrev State2 := Fin 4 → ℂ
19

```

Lean Gate Semantics

The semantics is defined directly on amplitudes.

Examples:

X

reads the amplitude corresponding to the basis where the target bit is flipped.

Z

adds a negative sign when the target bit is 1.

Hadamard

combines the two amplitudes:

$$\frac{\alpha_0 + \alpha_1}{\sqrt{2}}, \quad \frac{\alpha_0 - \alpha_1}{\sqrt{2}}$$

CNOT

flips the target when the control bit is true.

SWAP

exchanges two basis bits.

A circuit is an ordered sequence of these transformations.

What Lean Actually Proves

Lean checks exact state equalities for the fixed case studies:

- Bell;
- GHZ(3);
- Deutsch–Jozsa constant case;
- Deutsch–Jozsa balanced case.

Fixed obligations:

Bell, GHZ(3), DJ

General results:

unitarity/norm preservation, GHZ(n)

self-inverse gate laws

But the formal development also contains general theorems.

For every supported circuit:

- complex inner products are preserved;
- squared norm is preserved;
- normalized input states remain normalized.

And for GHZ there is a genuinely parametric result:

$$|0 \dots 0\rangle \mapsto \frac{|0 \dots 0\rangle + |1 \dots 1\rangle}{\sqrt{2}}$$

Why GHZ Is Important

The fixed GHZ(3) circuit is:

H(0)
CNOT(0,1)
CNOT(1,2)

and prepares:

$$|000\rangle \mapsto \frac{|000\rangle + |111\rangle}{\sqrt{2}}$$

This exercises a larger state space than Bell.

But a fixed three-qubit theorem is still only one instance, with chain $0 \rightarrow 1$, $1 \rightarrow 2$.

The project therefore also defines a parametric constructor:

H on qubit 0

+

fan-out CNOTs from qubit 0

and proves the result **inductively**, rather than enumerating 2^n basis states.

This is the clearest transition in the project from **verification by finite expansion to verification of a circuit family**.

Parametric GHZ Constructor vs GHZ 3

ParametricGhz.lean

```

/--
The first `count` fan-out gates for a fixed register.

The proof argument guarantees that targets `1, ..., count` are in range.
-/
def ghzFanout {n : Nat} :
  (count : Nat) → count + 1 ≤ n → Circuit n
| 0, _ => []
| count + 1, inRange =>
  ghzFanout count (by omega) ++
  [fanoutGate (count + 1) (by omega) (by omega)]

/-- The GHZ preparation circuit for every nonempty register size. -/
def ghzCircuit (n : Nat) (nonempty : 0 < n) : Circuit n :=
  [.h {0, nonempty}] ++ ghzFanout (n - 1) (by omega)

```

GeneratedGhz3.lean

```

11  /-- Einstein-Podolsky-Rosen, look at this :p -/
12  /-- Gate order is copied from the validated circuit IR. -/
13  def generatedGHZThreeQubitPreparationCircuit : Circuit 3 :=
14    [.h 0, .cnot 0 1 (by decide), .cnot 1 2 (by decide)]

```

Parametric GHZ Correctness Proof

```

311  /--
312  The parametric GHZ preparation theorem.
313
314  For every nonempty register, the generated circuit maps the all-zero basis ket
315  to `(ket 0...0 + ket 1...1) / sqrt(2)`.
316  -/
317  theorem ghz_correct (n : Nat) (nonempty : 0 < n) :
318    denote (ghzCircuit n nonempty) (ket (prefixBasis 0)) =
319    ghzState n := by
320    -- First expose sequential execution of H followed by the CNOT fan-out.
321    rw [ghzCircuit, denote_append]
322    change
323    denote (ghzFanout (n - 1) _)
324    (Gate.apply (.h {0, nonempty}) (ket (prefixBasis 0))) =
325    ghzState n
326    -- The two main lemmas now connect directly: H establishes `ghzPrefix 1`,
327    -- and fan-out grows it until every qubit belongs to the true branch.
328    rw [hadamard_zero_starts_ghz]
329    rw [ghzFanout_correct]
330    have fullRegister : n - 1 + 1 = n := by omega
331    rw [fullRegister]
332    rfl
333
334  /-- GHZ(3) follows directly from the parametric family theorem. -/
335  theorem ghz_three_correct :
336    denote (ghzCircuit 3 (by decide)) (ket (prefixBasis 0)) =
337    ghzState 3 :=
338    ghz_correct 3 (by decide)
339

```

Deutsch–Jozsa as a Stronger Regression Case

The project contains two fixed Deutsch–Jozsa examples with two query qubits:

Constant oracle: $f(x_0, x_1) = 0$

Balanced oracle: $f(x_0, x_1) = x_0$

Why include Deutsch–Jozsa?

- the states are less symmetric than Bell or GHZ;
- bit-ordering mistakes become visible;
- interference is essential to the algorithm;
- relative-phase mutations can preserve normalization while still making the result wrong;
- its asymmetric target exposes endianness mistakes.

Important limitation:

the project does not prove Deutsch–Jozsa for every promised oracle.

It proves two concrete instances.

Why Add Coq and SQIR?

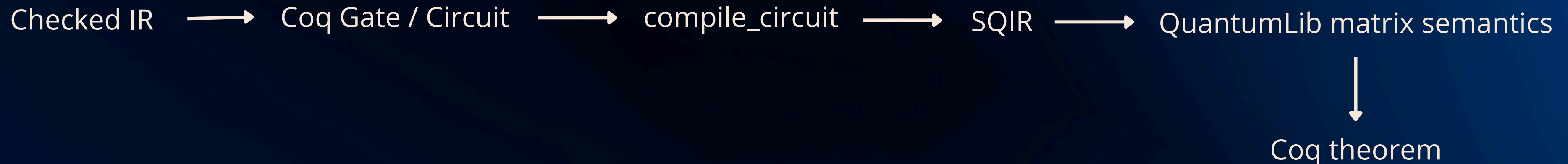
Lean answers:

Can we define a compact quantum semantics ourselves and prove properties directly?

Coq/SQIR answers a different question:

Can the same circuit description be translated into an established formal quantum IR?

The Coq backend therefore provides an independent formal path:

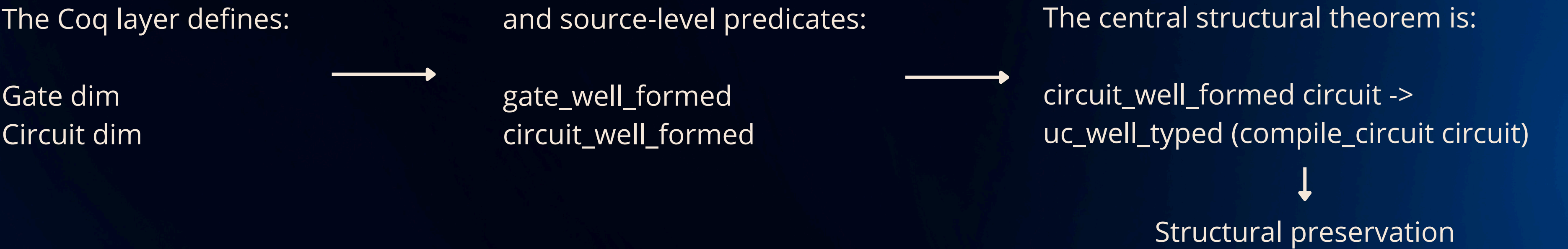


Lowering to SQIR

Circuit.v

```
45   Local Open Scope ucom_scope.  
46  
47   Definition compile_gate {dim : nat} (gate : Gate dim) : base_ucom dim :=  
48     match gate with  
49     | GateI target => ID target  
50     | GateX target => X target  
51     | GateZ target => Rz PI target  
52     | GateH target => H target  
53     | GateCNOT control target => CNOT control target  
54     | GateSWAP first second =>  
55       CNOT first second ; CNOT second first ; CNOT first second  
56   end.
```

What Coq/SQIR Proves



So a valid source circuit is translated into a well-typed SQIR circuit.

Generated correctness theorems then have the form:

$$run(C) \times |\psi_{in}\rangle = |\psi_{target}\rangle$$

→ Semantic obligation

for Bell, GHZ(3), and the two fixed Deutsch–Jozsa examples.

Structural preservation and semantic correctness are separate properties.

Compilation-Preservation Proof - Structural preservation

```

26 Definition Circuit (dim : nat) := list (Gate dim).
27
28 (* Keep source-level validity independent of SQIR so generated circuits expose
29    exactly the bounds and distinctness guarantees supplied by checked IR. *)
30 Definition gate_well_formed {dim : nat} (gate : Gate dim) : Prop :=
31   match gate with
32   | GateI target
33   | GateX target
34   | GateZ target
35   | GateH target => (target < dim)%nat
36   | GateCNOT control target
37   | GateSWAP control target =>
38     (control < dim)%nat /\ (target < dim)%nat /\ control <> target
39   end.
40
41 (* A positive dimension also makes the empty circuit's SKIP compilation valid. *)
42 Definition circuit_well_formed {dim : nat} (circuit : Circuit dim) : Prop :=
43   (0 < dim)%nat /\ Forall gate_well_formed circuit.
44

```

```

81 Theorem circuit_well_formed_compile_preservation {dim : nat}
82   (circuit : Circuit dim) :
83   circuit_well_formed circuit -> uc_well_typed (compile_circuit circuit).
84 Proof.
85   intros [positive_dim valid].
86   induction circuit as [| gate remaining IH].
87   - cbn [compile_circuit]. apply uc_well_typed_ID. exact positive_dim.
88   - inversion valid as [| ? ? gate_valid remaining_valid]; subst.
89     destruct remaining as [| next tail].
90     + cbn [compile_circuit]. apply compile_gate_well_typed. exact gate_valid.
91     + cbn [compile_circuit]. apply WT_seq.
92       * apply compile_gate_well_typed. exact gate_valid.
93       * apply IH. exact remaining_valid.
94 Qed.

```

Testing the Verification Infrastructure

Python

- parsers and validators;
- contract checks;
- IR conversion;
- generation tests.

Cross-system gate semantics

- every supported gate;
- registers up to three qubits, with backend-specific bounded coverage;
- valid operand placements;
- reversed CNOT roles;
- non-adjacent operands;
- spectator qubits.

Exhaustive finite basis/placement regressions within bounded dimensions

Mutated rewrite traces are rejected by deterministic replay

Negative tests intentionally introduce incorrect relative phases and require the theorem prover to reject them. CI also checks that regenerated Lean and Coq artifacts do not drift from the committed versions.

Verification Manifest and Traceability

Every verification run can produce a provenance manifest

It records:

- hashes of the exact IR and contract inputs;
- hashes of generated certificates and proof artifacts;
- numerical verification results;
- transpilation-equivalence results;
- Python / Qiskit / Lean / Coq toolchain versions;
- proof backend used;
- explicit proof status.

Crucial distinction

generated_not_kernel_checked

≠

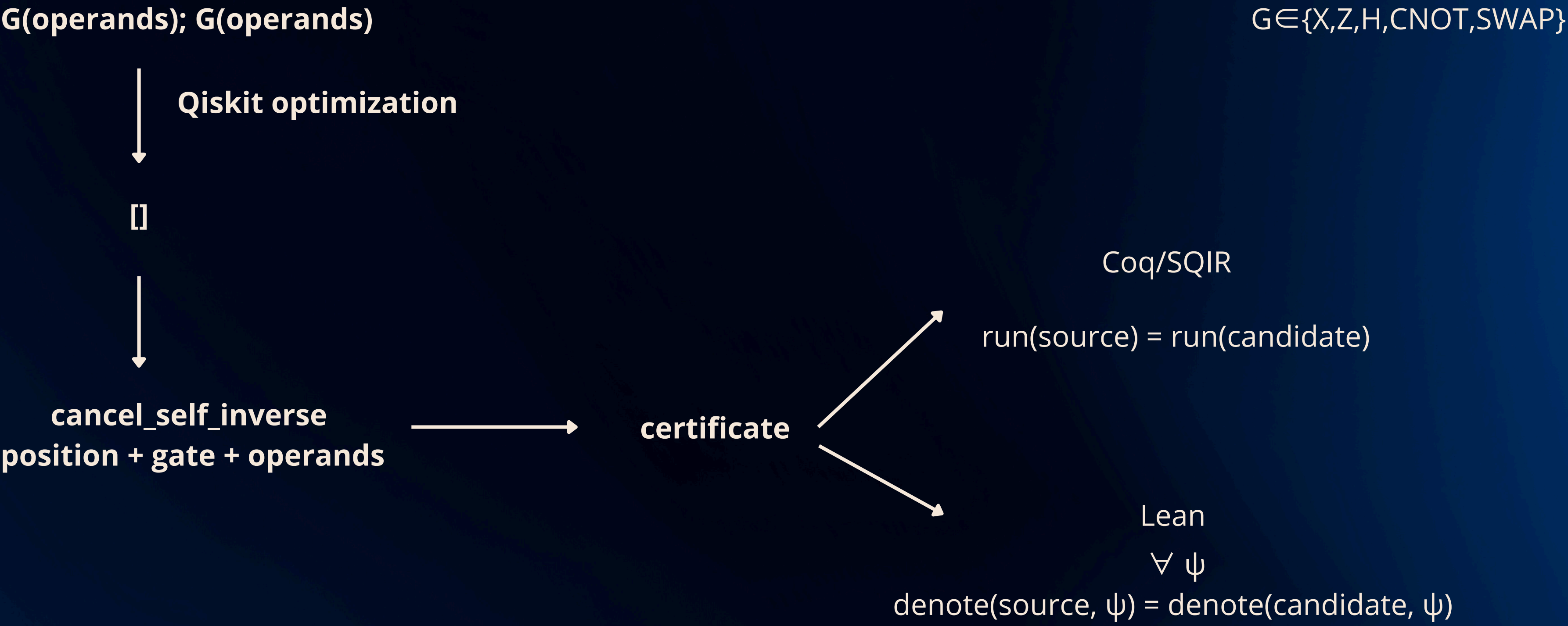
kernel-checked proof

The manifest binds together:

inputs $\longrightarrow$ verification results $\longrightarrow$ generated artifacts $\longrightarrow$ toolchain $\longrightarrow$ proof status

This improves reproducibility, auditability and traceability without pretending that artifact generation itself is a proof.

Certified Self-Inverse Rewrite



Two Different Styles of Formal Verification

Lean native semantics

Gate $\rightarrow$ State $n \rightarrow$ State n

dependent indices, induction, general GHZ theorem.

Coq/SQIR semantics

Gate $\rightarrow$ compile_gate $\rightarrow$ SQIR $\rightarrow$ uc_eval

established library, matrix semantics, lowering preservation.

Two independent semantic paths, not two syntaxes for the same proof

What Is Proved, and What Is Trusted?

Machine checked

- Lean formal circuit semantics;
- Bell, GHZ(3), and fixed DJ target equalities;
- parametric GHZ theorem;
- Lean norm and inner-product preservation;
- Coq/SQIR structural preservation;
- generated Coq semantic obligations.
- self-inverse cancellation equivalence in Lean and Coq/SQIR

Tested / numerically checked

- Qiskit execution;
- measurement probabilities;
- transpilation equivalence;
- cross-system gate correspondence.

Still trusted

- Qiskit;
- Python;
- Qiskit $\rightarrow$ IR extraction;
- IR $\rightarrow$ Lean generation;
- IR $\rightarrow$ Coq generation;
- correctness of the mathematical gate models relative to the real external system.

Negative Tests: Verification Must Also Fail

correct target → accepted

same probabilities, wrong relative phase → rejected

valid trace → obligation generated; CI requires kernel acceptance

mutated gate / position / operands → rejected before formal generation

What Changed from the Initial Research?

Initial idea

Qiskit → Contract → Lean

Final architecture Qiskit ↔ Checked IR



Narrower algorithmic scope, stronger verification architecture

Results and Limitations

Results

- practical Qiskit circuits connected to formal specifications;
- shared validated IR;
- executable contract checking;
- complete-operator transpilation evidence;
- Lean and Coq/SQIR proof generation;
- exact formal proofs for representative circuits;
- parametric GHZ verification;
- automated CI and negative tests.
- exact certificates for adjacent equal self-inverse gate cancellation in Lean and Coq/SQIR

Limitations

IR 0.1 does not yet model:

- measurement;
- reset;
- parameters;
- classical control;
- physical layouts;
- noise;
- formal transpilation currently supports one rewrite class only: self-inverse cancellation

Fixed-size generated proofs also scale exponentially when basis states are explicitly enumerated.

Possible Future Work

1. Verified translation

Prove that the Qiskit $\rightarrow$ IR and IR $\rightarrow$ formal translations preserve semantics.

2. Richer intermediate representation

Move toward a linear/resource-aware IR that can represent quantum ownership explicitly.

3. Measurements and classical control

Extend the semantics beyond purely unitary circuits.

4. More parametric algorithms

Replace finite enumeration with reusable lemmas and inductive proofs.

5. Broader certified compilation

Expand the rewrite catalogue beyond self-inverse cancellation and certify compositions of heterogeneous rewrites, eventually reduce/remove Python from the trusted path

Long-term direction

Move from:

proof of generated formal artifact

Toward:

proof of the complete source-to-formal pipeline

Conclusion

Quantum software validation requires multiple complementary notions of correctness.

Testing

- provides concrete evidence.

Contracts

- make expectations explicit.

Equivalence checking

- protects circuit transformations.

Formal semantics

- gives mathematical meaning to programs.

Lean and Coq

- allow those properties to become machine-checked theorems.

The main contribution of Formal Quantum Validation is therefore not a new quantum algorithm.

For one restricted rewrite class, cancellation of adjacent equal self-inverse gates, Lean proves all-input equivalence and Coq/SQIR proves complete-operator equality.

It is a prototype showing how:

Executable quantum software ↔ **formal specification** ↔ **machine-checked proof**
can be connected in one small and inspectable workflow.

References

[1] E. W. Dijkstra, Notes on Structured Programming , 1970.	[6] K. Hietala, R. Rand, S.-H. Hung, X. Wu and M. Hicks, A Verified Optimizer for Quantum Circuits , POPL 2021, DOI: 10.1145/3434318.
[2] A. Javadi-Abhari et al., Quantum Computing with Qiskit , 2024, arXiv:2405.08810.	[7] K. Hietala, R. Rand, S.-H. Hung, L. Li and M. Hicks, Proving Quantum Programs Correct , ITP 2021, DOI: 10.4230/LIPIcs.ITP.2021.21.
[3] L. de Moura and S. Ullrich, The Lean 4 Theorem Prover and Programming Language , CADE 28, 2021.	[8] inQWIRE, SQIR — A Small Quantum Intermediate Representation , software repository.
[4] The mathlib Community, mathlib: The Lean Mathematical Library .	[9] inQWIRE, QuantumLib — Coq Library for Reasoning about Quantum Programs , software repository.
[5] The Coq Development Team, The Coq Proof Assistant Reference Manual .	[10] L. Chiaretti, Formal Quantum Validation , source-code repository, 2026.

Thanks for the attention